
Madár Patrik

Rejtett Markov modellek alkalmazása

Markov láncok c. jegyzet alapján

Budapest, 2026

Contents

1	Az implementációról	3
1.1	Bevezetés	3
1.2	Valószínűségelméleti megfontolások	4
2	Implementáció	5
2.1	Inicializáció	5
2.2	Baum-Welch algoritmus	8
3	Alkalmazás	13
3.1	Eredmények	13
3.2	Egyéb alkalmazások	14

Chapter 1

Az implementációról

1.1 Bevezetés

Ebben a jegyzetben a rejtett markov modellek elméletének gyakorlatba ültetéséről lesz szó. A Baum-Welch algoritmus robusztussága kerül bemutatásra, alkalmas kezdeti értékből indítva. Az eredményekből kiderül, többdimenziós eloszlásokat tekintve a megadott inicializációval néhány lépéssel közel kerül az optimumhoz, összehasonlításképpen egyszerű iterációval több ezer véletlenszerű helyről indítva sem kapunk jobbat.

Az algoritmus felépítése a Markov láncok jegyzet szerkezetével van összhangban, használja az ott bemutatott diszkrét rejtett markov modellt, a stacionárius eloszlás létezését, annak kiszámítási módját, illetve a Baum-Welch/Forward-Backward algoritmusok egyéb aspektusait, például az alulcsordulás elkerülése érdekében skálázott értékekkel dolgozik, ügyel a kovarianciamátrixok szingularitására.

A jegyzet EM algoritmusról szóló részében tárgyalt iterációs móddal való összehasonlításképp kiderül, diszkrét esetben gyakran érdemesebb az inicializációt elvégezni, egyetlen algoritmus lefuttatásával, a több helyről indított keresési eljárással szemben.

Az eredmény – a modell paraméterein túl –, olyan kérdésekre ad választ, mint mennyi a következő napi várható konkrét hozam, következő napi hozamvektor és kockázati mátrix, illetve adott állapotokban a várható tartózkodási idő.

1.2 Valószínűségelméleti megfontolások

Piaci rezsimiek állapotgráfjaihoz tartozó Markov láncok gyakran rendelkeznek az ergodikus tulajdonsággal, így alkalmazható rájuk a jegyzetben tárgyalt ergodikus tétel, miszerint a π kezdeti eloszlás A átmenetmátrix esetén teljesíti a

$$\pi = \pi \cdot A$$

azonosságot, ez a

$$\sum_i (\pi_i) = 1, \pi_i \geq 0$$

feltételekkel egyértelműen meghatározza azt.

Adott rejtett állapotokhoz tartozó várható értékek a nagy számok törvénye szerint jól közelíthetők a tanítási részben számolt átlagokkal, ebből a kovarianciamátrixokra is jó közelítés adható.

Élve a feltételezéssel, hogy a piaci árfolyamok egymástól független valószínűségi változók összegéből keletkeznek, a centrális határeloszlás-tétel miatt közelíthetők normális eloszlással. Ismert, hogy normális eloszlást a várható értéke és kovarianciamátrixa egyértelműen meghatározza, ennek függvényében a megfigyelt állapotokba menő kibocsátási valószínűségek a megfelelő rejtett állapotokból menő paraméterekkel rendelkező normális eloszlásban felvett értékeknek felelnek meg, tehát ha a t . megfigyelés X_t , és a j . rejtett állapot várható értéke és kovarianciája μ_j, σ_j , akkor a kibocsátási valószínűség

$$f(X_t | \mu = \mu_j, \sigma = \sigma_j)$$

érték, ahol f a normális eloszlás sűrűségfüggvénye.

A kód jelenlegi formájában (könnyen módosítható) a megfigyelések az árfolyamváltozások százalékos értékei, a rejtett állapotok az adott mozgások irányát reprezentálják, azaz csökkenő/stagnáló/növekvő együttes állapotokat.

Chapter 2

Implementáció

2.1 Inicializáció

A kezdeti inicializáció több árfolyam együttes vizsgálata esetén az érdes eloszlás ellenére képes az optimumhoz közel kerülni néhány lépésben. Ezzel szemben ha véletlen paraméterekkel indítjuk az algoritmust, legtöbbször teljesen más lokális szélsőértékekbe szorul futtatás során, ahonnan a Markov láncok jegyzetben igazolt monoton növekvés miatt képtelen kiszabadulni (ugyan előfordulhat hogy körülötte ugrál, mivel a szingularitás elkerülése érdekében nem a pontos kovarianciákat használjuk).

A Baum-Welch első futtatásához szükségünk van a rejtett állapotok A átmenetmátrixára, π kezdeti eloszlására és B kibocsátási valószínűségek mátrixára. Pandas DataFrame adattípusokkal dolgozunk. A kód egy konkrét példával kerül szemléltetésre.

```
1 def load_data(currencies: list[str], start: int, end: int) -> list[
2     pd.DataFrame]: #currency = "BTC-USD" for example
3
4     try:
5         today = datetime.now()
6         train_start = (today - timedelta(days = start)).strftime("%Y
7             -%m-%d")
8         train_end = (today - timedelta(days = end)).strftime("%Y-%m
9             -%d")
10
11         #We get a raw data, because it is possible to be void lines
12         #for some dates
13
14         raw_datas = [yf.Ticker(currency).history(start = train_start
15             , end = train_end).ffill().bfill() for currency in currencies] #
16         #stored as a list of pandas DataFrame objects
17
18         #From here, we can use any pandas DataFrame module
19
20         common_index = raw_datas[0].index
21
22         for df in raw_datas[1:]:
23             common_index = common_index.intersection(df.index)
24
25         if len(common_index) == 0:
26             return "There are no common valid dates between given
27             currencies."
28
29         datas = [df.loc[common_index] for df in raw_datas]
30         return datas
```

```

24
25     except Exception as e:
26         print(f"Invalid input given: {e}.")

```

A bemenet egy sztringekkel kitöltött lista, ezek a sztringek a yahoo financehez köthetők, előbbi függvénynek a feladata hogy a nekik megfelelő pandas DataFrameek listáját adja vissza (a yf.Ticker pandas DataFrameet ad vissza). Azokhoz a dátumokhoz tartozó értékeket amiknél bármelyik árfolyamnak hiányossága van, elvetjük (kiegészítés is opció).

A függvény start és end paraméterei a yahoo financenek mondják meg milyen időintervallumban kéri le az adatokat, amiből a DataFrame készül. Ez egyrészt a betanítást, másrészt a futtatást segíti elő. A betanítási idő a futtatási időnek legalább kétszerese, például $start = 90$ és $end = 30$ értékekkel végezve a tanítást, a futtatásnak 30 napot hagyunk, ami éppen a fele.

A következő függvény a rejtett állapotokat inicializálja, létrehozva a közös állapotokat, eldönti melyik állapot melyikhez tartozik a DataFrameben, egy ennek megfelelő dataframe-t ad vissza:

```

1 def calculate_states(data: list[pd.DataFrame], n_states: int) -> pd.
  DataFrame:
2
3     if len(data) == 0 or not isinstance(data[0], pd.DataFrame):
4         raise ValueError("Invalid input given.")
5
6     df = pd.DataFrame(index=data[0].index) # Suppose all dataframes
  have the same index, we can use the index of the first dataframe
7
8     for i in range(len(data)):
9         if data[i].empty:
10             print(f"DataFrame at index {i} is empty. Please check
  the input data.")
11             return
12
13     df[f'Change {i}'] = (data[i]['Close'] - data[i]['Open']) /
  data[i]['Open'] * 100
14
15     conditions = [
16         df[f'Change {i}'] < -1, #Bear
17         (df[f'Change {i}'] >= -1) & (df[f'Change {i}'] <= 1), #
  Stagnate
18         df[f'Change {i}'] > 1 #Bull
19     ]
20     df[f'State {i}'] = np.select(conditions, list(range(n_states
  ))) # 0 = Bear, 1 = Stagnate, 2 = Bull
21
22     change_cols = [f'Change {i}' for i in range(len(data))]
23     df['Change'] = df[change_cols].values.tolist()
24
25     state_cols = [f'State {i}' for i in range(len(data))]
26     df['State'] = df[state_cols].values.tolist()
27
28     print(df.head()[['Change', 'State']])
29
30     return df

```

További kódrészlet az A átmenetmátrix és B kibocsátási mátrix inicializálását mutatja be, az előző fejezet megfontolásai szerint:

```

1 def initialize_transition_matrix(df: pd.DataFrame, n: int, dim: int)
  -> np.ndarray:
2

```

```

3     n_states = n**dim
4     elements = list(range(n))
5     vectors = list(itertools.product(elements, repeat = dim))
6     vectors = [list(x) for x in vectors]
7
8     A = np.zeros((n_states, n_states)) + 1e-6 # to avoid division by
9         zero, we add a small value to the matrix
10
11     for index1, i in enumerate(vectors):
12         for index2, j in enumerate(vectors):
13             for k in range(len(df['State'])):
14                 if k > 0 and df['State'].iloc[k] == j and df['State'
15 ].iloc[k-1] == i:
16                     A[index1, index2] += 1
17
18     row_sums = A.sum(axis = 1, keepdims = True)
19
20     A = np.divide(A, row_sums, where = row_sums != 0, out = np.
21         zeros_like(A))
22
23     return A
24 #Initializing the emission matrix B
25
26 def initialize_means_covs(df: pd.DataFrame, n1: int, dim1: int) ->
27     tuple[np.ndarray, np.ndarray]:
28
29     n1_states = n1**dim1
30
31     elements = list(range(n1))
32     vectors = list(itertools.product(elements, repeat = dim1))
33     vectors = [list(x) for x in vectors]
34
35     means = np.zeros((n1_states, dim1))
36     covs = np.zeros((n1_states, dim1, dim1))
37
38     for index, vector in enumerate(vectors):
39         mask = df['State'].apply(lambda x: x == vector)
40         state_datas_matrix = np.array(df[mask]['Change'].tolist())
41         if len(state_datas_matrix) > 1:
42             means[index] = np.mean(state_datas_matrix, axis = 0)
43             covs[index] = np.cov(state_datas_matrix, rowvar = False)
44             print(f"State: {vector}, Mean: {means[index]},
45 Covariance: {covs[index]}")
46         else:
47             means[index] = np.zeros(dim1)
48             covs[index] = np.eye(dim1)
49
50     return means, covs
51
52 def initialize_distribution(A: np.ndarray) -> np.ndarray: #
53     stationary distribution, by a theorem, we can find the
54     stationary distribution by solving the equation  $\pi * A = \pi$ ,
55     where  $\pi$  is the stationary distribution vector
56
57     K = A.shape[0]

```

```

54     M = A.T - np.eye(K)
55     M[-1, :] = 1
56
57     b = np.zeros(K)
58     b[-1] = 1
59
60     pi = np.linalg.solve(M, b)
61     return pi
62
63
64 def initialize_emission_matrix(X: np.ndarray, means: np.ndarray,
65                                covs: np.ndarray) -> np.ndarray:
66
67     X = np.asarray(X)
68
69     T, D = X.shape #T and D as time and dimension
70     n_states = means.shape[0] # number of hidden states
71
72     B = np.zeros((T, n_states))
73
74     for j in range(n_states):
75         B[:, j] = multivariate_normal.pdf(X, mean = means[j], cov =
76         (covs[j] + 1e-6 * np.eye(D)))
77
78     #Previous cycle does the following in a more optimal way:
79     #for i in range(T):
80     #    for j in range(n_states):
81     #        B[i, j] = multivariate_normal.pdf(X[i], mean = means[j], cov = covs[j])
82
83     return B

```

2.2 Baum-Welch algoritmus

A jegyzet hátralévő részében az algoritmus Expectation és Maximization lépését követjük le. A paraméterek számítása, annak indoklása röviden ismertetésre kerül, további indoklás a Markov láncok jegyzetben található.

Az Expectation lépésben az új

$$\alpha, c_\alpha, \beta, \gamma, \xi$$

(itt α, β a Forward-Backward paraméterei, c_α a súlyozási faktor, ami az algoritmus skálázott változatához szükséges, azt használjuk az alulcsordulás elkerüléséhez) paraméterek kerülnek inicializálásra.

```

1 def forward_scaled(A: np.ndarray, B: np.ndarray, pi: np.ndarray) ->
2     tuple[np.ndarray, np.ndarray]:
3
4     T, N = B.shape
5
6     alpha = np.zeros((T, N))
7     c_alpha = np.zeros(T) # scaling factors
8
9     alpha_hat_0 = pi * B[0]
10    c_alpha[0] = 1 / (np.sum(alpha_hat_0) + 1e-300)
11    alpha[0] = alpha_hat_0 * c_alpha[0]
12
13    for t in range(1, T):
14        alpha_hat_t = (alpha[t-1] @ A) * B[t]

```

```

14         c_alpha[t] = 1 / (np.sum(alpha_hat_t) + 1e-300)
15
16         alpha[t] = alpha_hat_t * c_alpha[t]
17
18     return alpha, c_alpha
19
20 def backward_scaled(A: np.ndarray, B: np.ndarray, c_alpha: np.
    ndarray) -> np.ndarray:
21
22     T, N = B.shape
23
24     beta = np.zeros((T, N))
25     beta[T-1] = c_alpha[T-1]
26
27     for t in range(T-2, -1, -1):
28         beta_hat_t = A @ (B[t+1] * beta[t+1])
29
30         beta[t] = beta_hat_t * c_alpha[t]
31
32     return beta
33
34 def gamma_belief(alpha: np.ndarray, c_alpha: np.ndarray, beta: np.
    ndarray) -> np.ndarray:
35
36     T, N = alpha.shape
37     gamma = np.zeros((T, N))
38
39     for t in range(T):
40         gamma[t] = alpha[t] * beta[t] / c_alpha[t] # bel[t]
41     return gamma
42
43 def ksi_belief(alpha: np.ndarray, c_alpha: np.ndarray, beta: np.
    ndarray, A: np.ndarray, B: np.ndarray) -> np.ndarray:
44
45     T, N = alpha.shape
46
47     ksi = np.zeros((T-1, N, N))
48
49     for t in range(0, T-1):
50         for i in range(N):
51             for j in range(N):
52                 ksi[t, i, j] = alpha[t, i] * A[i, j] * B[t+1, j] *
                    beta[t+1, j] * c_alpha[t+1]
53
54     return ksi

```

A Maximization részben a megfigyeléseinket és az újonnan számított paramétereket használjuk az A mátrix, rejtett állapotokhoz tartozó várható értékek/kovarianciák és a B mátrix újrabecslésére: $\xi_t(i, j)$ -vel jelölve az $i \rightarrow j$ állapotátmenet valószínűségét a t időpontban, $\gamma_t(i)$ -vel az i -ben lét valószínűségét a t időpontban,

$$\sum_t \xi_t(i, j)$$

az $i \rightarrow j$ átmenetek várható értéke,

$$\sum_t \gamma_t(i)$$

az i -ből menés várható értéke, ebből kapjuk az új $A = (a_{ij})_{i,j}$ átmenetmátrixot:

$$a_{ij} = \frac{\sum_{t=1}^{T-1} \xi_t(i, j)}{\sum_{t=1}^{T-1} \gamma_t(i)}$$

Hasonlóan, ha csak azon t időközön összegezzük $\gamma_t(i)$ -t, ahol v_k értéket vett fel, megkapjuk az i létben v_k megfigyelések várható értékét, ebből

$$b_i(k) = \frac{\sum_{Y_t=v_k}^T \gamma_t(i)}{\sum_{i=1}^T \gamma_t(i)}$$

A várható értékeket és szórásokat súlyított átlagként kapjuk:

$$\mu_j = \frac{\sum_{t=1}^T \gamma_t(j) \cdot X_t}{\sum_{t=1}^T \gamma_t(j)},$$

illetve

$$\Sigma_j = \frac{\sum_{t=1}^T \gamma_t(j) \cdot (X_t - \mu_j) \cdot (X_t - \mu_j)^T}{\sum_{j=1}^T \gamma_t(j)}$$

```

1 def recalibrate_transition_matrix(ksi: np.ndarray, gamma: np.ndarray
2 ):
3     T, N = gamma.shape
4     A_new = np.zeros((N, N))
5
6     gamma_sum = np.sum(gamma[:-1], axis = 0)
7
8     for i in range(N):
9
10         if gamma_sum[i] < 1e-12:
11             A_new[i, :] = 1.0 / N
12         else:
13
14             for j in range(N):
15                 #(expected number of transitions from S_i to S_j) /
16                 (expected number of transitions from S_i)
17                 A_new[i, j] = np.sum(ksi[:, i, j], axis = 0) /
18                 gamma_sum[i]
19
20     A_new = np.nan_to_num(A_new, nan = 1.0/N)
21     A_new = A_new / np.sum(A_new, axis=1, keepdims=True)
22
23     return A_new
24
25 #Before recalibrating B, we need to recalibrate means and covs.
26
27 def recalibrate_means_covs(X: np.ndarray, gamma: np.ndarray) ->
28 tuple[np.ndarray, np.ndarray]:
29
30     X = np.asarray(X)
31
32     T, D = X.shape
33     n_states = gamma.shape[1]
34
35     means_new = np.zeros((n_states, D))
36     covs_new = np.zeros((n_states, D, D))

```

```

34
35 gamma_sum_total = np.sum(gamma, axis = 0) # Averages times spent
    there for every state
36
37 min_cov = 1e-6
38
39 for j in range(n_states):
40
41     if gamma_sum_total[j] < 1e-6:
42         covs_new[j] = min_cov * np.eye(D)
43         continue
44
45     means_new[j] = np.sum(gamma[:, j, np.newaxis] * X, axis = 0)
    / gamma_sum_total[j]
46
47     diff_j = X - means_new[j]
48     cov_j = np.zeros((D, D))
49
50     cov_j = np.einsum('t, ti, tj -> ij', gamma[:, j], diff_j,
    diff_j) / gamma_sum_total[j]
51     #alternatively covs_new[j] = ((diff_j * gamma[:, j, np.
    newaxis]).T @ diff_j) / gamma_sum_total[j] + 1e-6 * np.eye(D)
52
53     covs_new[j] = cov_j + min_cov * np.eye(D)
54
55     means_new = np.nan_to_num(means_new, nan=0.0, posinf=0.0, neginf
    =0.0)
56     covs_new = np.nan_to_num(covs_new, nan=0.0, posinf=0.0, neginf
    =0.0)
57
58     return means_new, covs_new
59
60 def recalibrate_emission_matrix(X: np.ndarray, means: np.ndarray,
    covs: np.ndarray) -> np.ndarray:
61
62     """
63     X: (T, D)
64     means: (n_states, D)
65     covs: (n_states, D, D)
66     """
67
68     T, D = X.shape
69     n_states = means.shape[0]
70
71     B_new = np.zeros((T, n_states))
72
73     for j in range(n_states):
74         B_new[:, j] = multivariate_normal.pdf(X, mean = means[j],
    cov = covs[j], allow_singular = True)
75
76     B_new = np.nan_to_num(B_new, nan = 1e-300, posinf=1.0, neginf=1e
    -300)
77     B_new[B_new == 0] = 1e-300
78
79     return B_new

```

Megvan tehát az Expectation és Maximization minden értékének számítására a függvény, ezt összerakva kapjuk az algoritmust:

```

1 def baum_welch_algorithm(X: np.ndarray, states: pd.DataFrame,

```

```

n_states: int, dim: int, has_initialization: bool, n_iter: int =
50):
2
3     X = np.asarray(X)
4
5     if has_initialization:
6         #Initializitaion:
7         A = initialize_transition_matrix(states, n_states, dim)
8         pi = initialize_distribution(A)
9         means, covs = initialize_means_covs(states, n_states, dim)
10
11     else:
12         A, pi, means, covs = generate_random_params(X, n_states)
13
14     B = initialize_emission_matrix(X, means, covs)
15
16     alpha, c_alpha = forward_scaled(A, B, pi)
17
18     log_likelihood = -np.sum(np.log(c_alpha + 1e-300))
19
20     #Expectation-Maximization cycle:
21
22     for iteration in range(n_iter):
23
24         #Expectation step:
25
26         alpha, c_alpha = forward_scaled(A, B, pi)
27
28         #To see converging log-likelihood values
29
30         log_likelihood = -np.sum(np.log(c_alpha + 1e-300))
31         #print(f"At iteration {iteration}, log-likelihood is {
log_likelihood}")
32
33
34         beta = backward_scaled(A, B, c_alpha)
35         gamma = gamma_belief(alpha, c_alpha, beta)
36         ksi = ksi_belief(alpha, c_alpha, beta, A, B)
37
38         #Maximization step:
39
40         A = recalibrate_transition_matrix(ksi, gamma)
41         means, covs = recalibrate_means_covs(X, gamma)
42         B = recalibrate_emission_matrix(X, means, covs)
43
44     return A, pi, means, covs, B, log_likelihood

```

Chapter 3

Alkalmazás

3.1 Eredmények

Az algoritmusnak kétféle futtatása lehetséges: inicializációs és inicializáció nélküli. Összehasonlítás gyanánt az inicializációsat egyszer futtatjuk, az inicializáció nélkülit saccolt értékekkel 1000-szer. Utóbbi számot növelve közelebb jutunk az értékhez ami körül az inicializációs bolyong, általában nem érjük el. Etalonként a log-likelihoodot használjuk.

A lépéseknél a kód ügyelt arra hogy ne kapjon szinguláris mátrixot, emiatt a monoton növekvő tulajdonság nem teljesül, egy idő beállta után -68 körül mozog a log-likelihood (eléri a -59-et is gyakran), ezzel szemben a generált értékekből indítottaknál 1000 futtatás után, darabonként 100 iterációval a legnagyobb likelihood nagyjából -71.

```
1 input = ["BTC-USD", "ETH-USD"]
2 iteration_number=110
3 optimums = []
4
5 datas = load_data(input, 90, 30)
6 merged_states = calculate_states(datas, 3) # Calculate states for
   Bitcoin and Ethereum with 3 states
7
8 observations = calculate_states(load_data(input, 30, 0), 3)["Change"
   ].values.tolist()
9
10 result = baum_welch_algorithm(
11     observations,
12     merged_states,
13     3,
14     len(input),
15     True,
16     7
17 )
18
19 print(result)
20
21 for _ in range(iteration_number):
22     a, pi, means, covs, b, opt = baum_welch_algorithm(
23         observations,
24         merged_states,
25         3,
26         len(input),
27         False,
28         1000
29     )
```

```

30     optimums.append(opt)
31
32 print(max(optimums))

```

3.2 Egyéb alkalmazások

A modell – a paraméterek számításán túl – alkalmas egyéb értékek meghatározására. Például a holnapi napra várható konkrét hozam meghatározása

$$\pi_{T+1} = \gamma_T \cdot A$$

kiszámításával történhet, ahol π_{T+1} a holnapi állapoteloszlás, γ_T a mai állapotok valószínűsége. Ezzel súlyozva a μ, Σ értékeket, kapjuk a

$$E(X_{T+1}) = \sum_{k=1}^N \pi_{T+1,k} \cdot \mu_k$$

holnapi várható hozamot, illetve a

$$Var(X_{T+1}) = \sum_{k=1}^N \pi_{T+1,k} \cdot (\Sigma_k + (\mu_k - E(X_{T+1}))(\mu_k - E(X_{T+1}))^T)$$

kockázati mátrixot.

Az átmenetmátrix ismeretében adott i állapotra az ott várhatóan eltöltött idő is megkapható, ha ugyanis a_{ii} az i -ből i -be kerülés valószínűsége, akkor $1 - a_{ii}$ az i -ből nem i -be kerülése. Erre geometriai eloszlás illeszthető, addig lépkedve mígnem eljön a kedvező eset, az i -ből nem i -be kerülés. Ezen geometriai eloszlás várható értéke

$$E(\text{napok száma az } i. \text{ állapotban}) = \frac{1}{1 - a_{ii}}$$